Evaluation of 22nm Memristor-Based Main Memory (MBMM) in Commodity DIMM Architectures

Yuval Kogan

For Prof. Shahar Kvatinsky

September 2026

The project book. Extended book, with the complete record: Technical_Report.pdf.

Abstract

Can memristor-based non-volatile memory replace DRAM in a commodity DIMM at 22nm? To answer that I built a cross-layer simulation pipeline that carries an NVSim device characterization of transistor-gated (1T1R) and selector-gated (1S1R) ReRAM, as single-level and multi-level cells, through NVMain 2.0 on six workload traces at one matched 2048 x 2048 array organization. **Latency:** at projected media timings the ReRAM DIMM is faster than a DDR5-4800 baseline on the three CPU traces at both ends of the write-pulse bracket, 41.4 to 48.3 ns for 1T1R SLC and 36.8 to 47.4 ns for 1S1R SLC against 83.1 ns under GCC (78.2 ns when the baseline runs the same close-page policy as the ReRAM configurations), and slower on the write-dominated AlexNet output-map burst by 1.5x to 3.0x for 1T1R SLC, with the counterweight that the identical DIMM built from published silicon timings averages 11.9 us and 379 us under GCC. **Power:** per gigabyte the module costs 12.7x to 17.4x DDR5 for the SLC tracks, a device-only figure and an upper bound, with gating break-even at 94.3% of the static component, which lies inside the 94 to 98% idleness band reported for web-serving deployments. **Endurance:** every lifetime here is a projection, and the wear leveling rather than the cell decides it, ideal uniform leveling at 64 GiB and 10^6 cycles per cell giving 38.6 years under GCC, 3.6 under LBM and 4.3 under STREAM. **Density:** the selector-gated cross-point reaches 1.24x DDR5's die-level density as SLC and 2.47x as MLC, against 0.36x and 0.73x for the transistor-gated cell, which never clears parity. Selector-gated 22nm SLC ReRAM is therefore a density play with competitive projected latency and a large, honest power deficit per gigabyte, while the transistor-gated variant matches it on leakage and loses on area; *neither is a drop-in DDR5 replacement at 22nm.*

1. Introduction

1.1. Why this question, now

Two pressures meet in commodity main memory. The first is economic. By late 2025 the reallocation of DRAM wafers to High-Bandwidth Memory for AI accelerators had opened a severe supply-demand gap for ordinary DIMM memory **[1], [2]**. The second is physical. DRAM's planar cell is held back by capacitor leakage and short-channel effects, and commodity DDR5 is fabricated on 10 nm-class DRAM nodes that sit at the end of the capacitor's scaling roadmap **[3]**. A resistive cell stores its state as resistance rather than as charge, so it has no capacitor to scale and needs no refresh.

The question this project asks is narrow and practical: can memristor-based non-volatile memory substitute for DRAM in a standard DIMM? One constraint shapes every answer to it. Writes physically wear the cell, so endurance is finite, and any viable DRAM replacement has to buy its lifetime architecturally rather than from the device.

To answer the question I built a cross-layer simulation pipeline that carries a device-level 22nm ReRAM characterization from NVSim **[4]** into cycle-accurate system simulation in NVMain 2.0 **[5]**, and I ran four ReRAM tracks, a JEDEC-sourced DDR5-4800 baseline, a legacy PCM baseline and two published-silicon configurations against six workload traces. Chapter 2 describes the pipeline, Chapter 4 reports what it measured.

1.2. ReRAM in one page

A memristor cell is a two-terminal metal-insulator-metal stack, typically an HfOx or TaOx oxide, whose electrical resistance is its stored state. A programming voltage grows or ruptures a conductive filament of oxygen vacancies through the oxide, switching the cell between a low-resistance and a high-resistance

state; a read applies a small sense voltage that leaves the filament undisturbed **[6]**. The filament persists without power, so storage is non-volatile. Filament formation is stochastic and writes stress the filament, which is why the resistance targets below are engineering choices and why endurance is the binding constraint of Chapter 4.

I standardized on a 22nm FinFET low-operating-power node, which is logic-compatible and has fabricated ReRAM precedent at that node **[7]**. It is a deliberately conservative frame: 22nm has several shrink generations of headroom ahead of it, so the density figures here are a floor on ReRAM's potential rather than a ceiling. Cells are calibrated to high resistance targets, $10^5\Omega$ in the low-resistance state and $10^9\Omega$ in the high, to hold a sensing margin against IR drop on long bitlines **[8]**. The arrays here are 2048 cells on a side, one doubling past that source's own 1024-cell design point, and the extended book shows the criterion still holds to within 1%.

In a crossbar, current can flow through unselected cells as well as the selected one. This sneak-path problem is what separates the two device structures compared here, and it is the whole of the density argument. **1T1R** puts a CMOS access transistor in series with each cell, which eliminates sneak paths completely and costs area: roughly $20F^2$ per bit on a transistor-drive-limited estimate. **1S1R** replaces the transistor with a two-terminal selector whose sharply non-linear threshold suppresses the sneak current instead, reducing the cell to an idealized $4F^2$ **[9]**. The selector is therefore what buys the density, and it is also what the density claim is conditional on. NVSim models no selector physics of its own, so whether a 2048-cell tile is actually sneak-path-viable is settled separately, by an analytic selector layer built on the crossbar read model of **[10]** (Chapter 4): at a present-day ovonic-threshold-switch quality it is not, and at the best published selector it is.

Each cell type is evaluated at one bit per cell (SLC) and at two (MLC). MLC puts twice the capacity behind the same chips and pays for it in write time. NVSim has no multi-level model, so the MLC tracks apply multipliers derived from the EMBER macro's own published measurements, 1.5x on read latency and 3.263x on write latency **[11], [12]**.

Two things are worth separating before any number appears. NVSim produces an array-core projection, and *no fabricated ReRAM part reads or writes in nanoseconds*. Micron and Sony's 27nm 16 Gb 1T1R part reads in 2.3 us and writes in 11.7 us **[13]**; SanDisk and Toshiba's 24nm 32 Gb 1S1R cross-point reads in 40 us and writes in 230 us **[14]**. Both are carried through this evaluation as their own configurations, on the same traces and the same module geometry, and they appear beside every projected latency.

The one non-volatile main memory ever shipped is worth naming here for its vocabulary as much as for its history. **3D XPoint** is the *media*: a phase-change storage element in series with an Ovonic Threshold Switch selector, that is, a 1S1R cross-point, announced by Intel and Micron in 2015 **[15]**, confirmed at the materials level by an independent physical teardown **[16]** and by Intel's own device paper **[17]**. **Optane** was the *product* built on that media, and Intel exited the business in 2022 **[18]**. Optane's published latencies were measured on a shipped module **[19]** and are therefore *device* numbers: they include a controller, ECC and a DIMM interface. Every ReRAM latency in this book is a *media* number, produced by a device model driving a memory simulator. The two are never the same kind of measurement and are never compared as though they were.

1.3. What this book claims, and how to read it

The claim is a balanced verdict, not an advocacy case. Selector-gated 22nm SLC ReRAM is a density play with competitive projected latency and a large, honest power deficit per gigabyte; the transistor-gated

variant matches it on leakage and loses on area; write endurance is the binding constraint and wear leveling, not the cell, decides it. The extended book states the conclusion in one line, and this book keeps it: *neither is a drop-in DDR5 replacement at 22nm.*

One caveat governs every latency in this book and is stated here once, in full, so that later chapters can refer to it rather than repeat it.

> **The projection caveat.** First, every ReRAM timing reported here is an NVSim projection of an array core, not a measurement of a fabricated part. Second, the same DIMM built from published silicon timings instead averages 11.9 us (1T1R) and 379 us (1S1R) under GCC and does not finish the LBM window at all, completing 18.2% and 1.0% of it. Third, on the write-dominated AlexNet output-map burst the projected DIMM is *slower* than DDR5, 353.1 ns for 1T1R SLC and 391.5 ns for 1S1R SLC against 234.3 ns. The latency verdict of this book therefore belongs to a projected device, and every reading of it is incomplete without its silicon counterweight.

Two further reading rules follow from the same discipline. Every ReRAM latency here is a media number in the sense of Section 1.2, and every Optane or CXL figure quoted in Chapter 5 is a device number. And every endurance figure in this book is a *projection* from a write distribution measured inside a simulation window, never a measured lifetime; the word “measured” is not used of any lifetime.

This is the book: the summary of the whole work, with every reference it rests on. The extended book, `Technical_Report.pdf`, is the complete record underneath it: the full methodology, every table in its uncondensed form, the simulation-fidelity audit, the reproduction instructions and an appendix recording claim by claim what changed since the September 3 version.

2. Method

2.1. The pipeline

No single tool spans the distance from an oxide stack to a DIMM latency, so I built a pipeline that does. NVSim **[4]** characterizes exactly **one 1 Gb chip**: read and write latency in nanoseconds, energy per access, leakage and die area. `2_extract_hardware_metrics.py` pulls those figures into JSON and applies the MLC penalty model. `3_gen_nvmain_config.py` turns the JSON into an NVMain configuration, mapping the device read latency once across tRCD plus tCAS and the cell write latency onto write recovery, and validating both splits against the rendered configuration text. NVMain 2.0 **[5]** replicates that chip 1, 8, 16 or 64 times, wraps it in its own controller, decoder and interconnect and replays a trace against it cycle by cycle. `process_metrics.py` turns the resulting statistics into the CSVs every table here is drawn from (Figure 1).

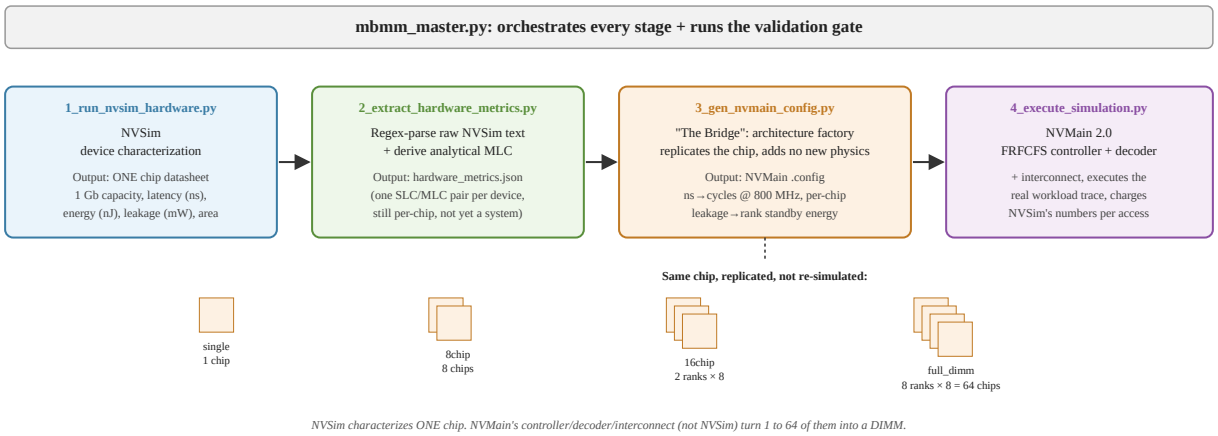


Figure 1: the pipeline end to end. NVMain never re-touches cell-level physics.

`mbmm_master.py` orchestrates the whole sequence. It cleans build artifacts, recompiles both C++ engines, runs a sanity simulation, then executes characterization, extraction, configuration generation, simulation and post-processing as one gated flow: no result is accepted that did not come through it. It takes the replay window in nanoseconds and converts it to each technology's own cycle budget from that technology's own clock, so every configuration replays the same wall-clock window.

The results in this book come from frozen datasets, not from a live working directory: a 120-run primary matrix of twenty configurations against six workloads, plus seven sensitivity series that each change one flag, a 1024-organization axis and a one-channel control run, every one of them carrying a manifest of its flags and provenance columns on every processed row. The controller's queue depth is one of those axes, and the write-queue sizing question it answers is reported in Section 4.4. The Python layer is covered by a test suite, 783 tests at this revision's closing verification, and by a regression harness that re-runs a fixed configuration against a stored reference. Both are part of the gate: a pipeline change is not considered to have worked until the suite, the harness and `mbmm_master.py` all pass.

2.2. Device characterization

Both cell types are characterized at *one matched array organization*, 2048 x 2048 subarrays at a sense-amp column mux of 64, forced identically for 1T1R and 1S1R and verified against NVSim's own printed geometry on every run. This is the single most consequential methodological choice in the evaluation, and Chapter 3 explains why. Table 1 is the characterization the system simulation inherits.

Table 1: NVSim device-level characterization per 1 Gb chip (22nm FinFET LOP, 1.4 V ReadVoltage nominal), at the forced matched organization of 2048 x 2048 subarrays at sense-amp mux 64.

Track	Cell size	Read lat. (ns)	Write lat. (ns)	Leakage (mW/chip)	Die area (mm²)
1T1R SLC	20F²	10.12	15.26	108.384	12.008
1T1R MLC	20F²	15.18	49.79	108.384	12.008
1S1R SLC	4F²	4.70	24.59	108.384	3.540
1S1R MLC	4F²	7.05	80.23	108.384	3.540

The one device-level advantage that survives a matched organization is the selector's die area, 3.540 against 12.008 mm² per 1 Gb chip; leakage is identical for both cell types, for the reason lesson 1 of Chapter 3 gives. The 1S1R rows are uncalibrated against any fabricated part. Source: results/rev2026-09_primary_csv/hardware_metrics.json.

NVSim has no multi-level-cell model, so the MLC rows apply an analytic penalty on top of the SLC characterization: 1.5x read latency and 3.263x write latency, with twice the capacity behind the same chips. Both multipliers are derived by the same bandwidth-ratio method from the EMBER macro's own published measurements, the journal report **[11]** and its conference precursor **[12]**; leakage and die area are unchanged, because the array is the same array.

2.3. Baselines

DDR5-4800. The primary baseline is a conventional 16 GiB DDR5-4800 RDIMM built from eight 16 Gb x8 devices, modeled with DDR5's defining architectural feature: two independent 32-bit subchannels per DIMM, each with its own command bus and 32 banks, serving a 64-byte access as a 16-beat burst on its 32-bit bus. Its timings are the JESD79-5 values for the 4800B bin **[20]**, read through a vendor's transcription (Micron's DDR5 core data sheet) and two DDR5-4800 module datasheets, all of which agree; the JEDEC text itself is access-gated and was not consulted. The scope of that statement is exact: *every DDR5-4800 device timing is taken from JESD79-5, with three documented short-value choices that favor the baseline (tCCD, tWTR, tRRD) and a handful of NVMain-specific keys with no DDR5 counterpart; the extended book labels all 36 device and simulator keys with their provenance.* The core numbers are the 4800 MT/s bin at 1.1 V: tCAS-tRCD-tRP of 40-39-39, CWL of 38 cycles, tWR 30 ns, tRRD_S 8 cycles and tFAW 32 cycles, with an all-bank refresh every tREFI of 3.906 us occupying tRFC of 295 ns. The baseline keeps NVMain's open-page row-buffer policy (`ClosePage 0`), the value the configuration was introduced with, while every ReRAM configuration runs the relaxed close-page policy (`ClosePage 1`); Section 4.1 gives the baseline under both policies. A second disclosure belongs with it: the DDR5 power model uses SK hynix's published 16 Gb DDR5-4800 IDD limits, the lower of the two vendor tables consulted, and the two power-down currents hynix does not publish are scaled from Micron's. Datasheet IDD values are specification limits rather than typicals, with no typical DDR5 figure published to use instead, so the 0.797 W of Table 5 is a worst-case-current figure for the lower-current vendor, not a typical power.

PCM. A legacy non-volatile baseline, named for its architectural framing in Lee et al. **[21]**, is carried as the floor-power reference. It was never run through NVSim: its timing and energy constants are inherited unmodified from NVMain's own bundled example configuration and were not independently re-derived here. It runs at its cited 400 MHz derivation basis and on the same 250 ms matched window as everything else. Its configuration specifies no access-device topology, so it is a black-box timing and energy model rather than a characterization of any particular PCM cell, and the one PCM product ever shipped is, as Section 1.2 records, a selector-gated 1S1R cross-point that is not modeled here at all.

Published silicon. The two fabricated-timing configurations of Section 1.2 are generated directly from those data sheets and run on the same traces and the same module geometry, labeled 1T1R SILICON and 1S1R SILICON, and they appear beside every projected latency in Chapter 4. The extended book puts the reason plainly, and this book keeps it: *the two published-silicon rows sit in the same table deliberately: they are the counterweight to every projected row above them, and no reading of this table is complete without them.*

Access granularity. Every technology in this comparison is accessed at **64 bytes**. That is the x86-64 cache line, matched by 64 B lines on contemporary Arm server cores **[22]**; it is what DDR5's two 32-bit subchannels deliver at a burst length of 16 **[23]**; and it is what NVMain's address translator is hardwired to. The gem5 traces are 64 B aligned as they come. The SCALE-Sim traces are not: they are tensor-element granular and only 1.6% aligned, so for them the 64 B width is imposed by the configuration rather than carried by the trace. A dedicated 64 versus 128 B study, reported in the extended book, found 128 B worse on every axis.

The ReRAM interface. All ReRAM configurations run behind an 800 MHz (1600 MT/s) DIMM interface, and this is a modeling assumption that must be read as one. Only two protocol timings in a generated ReRAM configuration come from NVSim: tRCD plus tCAS, carrying the device read latency once, and tWR, carrying write recovery. Every other protocol timing is a constant in *interface cycles* and therefore scales in nanoseconds with whatever clock is chosen. The 800 MHz figure itself carries *no independent ReRAM-interface citation*: a targeted search found none, and the only datasheet-verified ReRAM chip I/O clocks located run 80 to 160x slower, actively contradicting rather than merely failing to support it. The figure is positioned only between two precedents, twice the 400 MHz basis of NVMain's PCM lineage and below the DDR4-2666 electrical interface that carried the DDR-T protocol on Intel's shipped Optane DIMMs. Chapter 4 quantifies what the assumption is worth by re-running the same media at 1333 and 2400 MHz.

2.4. Traces

Table 2 lists the six traces, from three sources. The three SPEC and STREAM traces were captured with gem5 **[24]** running the X86 out-of-order model with L1 and L2 caches enabled, after 500 M fast-forwarded instructions, with the first 10 ms of the detailed region discarded as cache warm-up; each reconciles exactly with gem5's own memory-controller request counters. The three AI traces come from the SCALE-Sim systolic-array simulator **[25]**.

Time base and window: one trace cycle is one NVMain cycle at CPUFreq 3000, that is three cycles per nanosecond, and every configuration replays the same 250 ms wall-clock window against the identical admitted request population.

Table 2: the six workload traces.

Trace	Source	Records	Read/write in window	Span in the 250 ms window
gcc_spec2017	gem5, SPEC CPU2017 502.gcc_r **[26]**	545,566	298,792 / 220,776 (58/42)	fills the window, sustained
lbm_spec2017	gem5, SPEC CPU2017 519.lbm_r **[26]**	10,779,425	3,179,900 / 2,384,804 (57/43)	fills the window, sustained
stream	gem5, STREAM **[27]**	48,439,056	3,996,254 / 1,998,124 (67/33)	fills the window, sustained
gpt2_ifmap	SCALE-Sim **[25]**	65,536	65,536 / 0 (pure read)	2.2 us burst
alexnet_layer1_ifmap	SCALE-Sim **[25]**	1,269,600	1,269,600 / 0 (pure read)	60.5 us burst
alexnet_layer1_ofmap	SCALE-Sim **[25]**	135,424	0 / 135,424 (pure write)	4.5 us burst

Records are trace totals from each trace's provenance sidecar; the in-window columns are what the 250 ms replay window admits, which for the three gem5 traces is a prefix of the trace. Each AI trace is a single microsecond burst delivered once inside the window, 2.2, 60.5 and 4.5 us respectively, so every rate derived from one is a burst rate and is marked as such wherever it is used. SCALE-Sim's `layer1` folder is zero-indexed, so the AlexNet traces are AlexNet's second convolution layer, Conv2; the label is kept for continuity.

A seventh workload was planned and is absent. 505.mcf_r could not be traced: gem5 panics inside the benchmark's own input reader about 0.13 ms into the detailed region, identically on two independent attempts, so mcf is parked rather than approximated and no mcf number appears anywhere in this book.

2.5. What was built

The pipeline as described above is not what I found. A systematic audit of the NVSim-to-NVMain flow, conducted against the raw simulator sources and statistics files rather than against their documentation, found fourteen *silent* failure modes: defects that produced plausible numbers rather than errors. Thirteen were repaired in this project's toolchain and the affected simulations re-run. Table 3 is the list. It records the mode and the fix only; the values each repair moved are tabulated in the extended book's change appendix.

Table 3: the thirteen repaired silent failure modes, in eleven rows, two pairs sharing one repair between them.

Failure mode	Repair
Efficiency metric mixed clock domains: watts times cycles from two different clocks	Every power-delay product computed in physical W·ns
The generator's leakage key was dead configuration, NVMain's source never reads it, and in its place generic standby defaults gave every non-volatile track the same static floor	NVSim leakage mapped onto the standby-energy parameters NVMain does read; static power is technology-specific for the first time
Reported "total system power" was the most-loaded rank, not the module	Whole-module sums for every technology
NVMain's power-down state machine was disabled in source: no idle policy could be simulated	Dormant call site restored; a shared mechanism, so it activated for every technology at once
Trace provenance: no region of interest, no warm-up, a parser keeping about four records per request	Every trace regenerated with caches, a documented region, a warm-up discard and a provenance sidecar
DDR5 refresh machinery inherited from a DDR3-era template, never rescaled to the DDR5 clock	Recalibrated to JESD79-5: tREFI 3.906 us, tRFC 295 ns
PCM ran at twice its cited clock basis; a typo zeroed its per-bit write energy	400 MHz basis restored in configuration and reporting layer; typo fixed
ReRAM access energies written under key names NVMain never reads	Live energy keys wired; the ReRAM matrix re-run
Heterogeneous host-CPU frequencies gave each technology a different offered load	One host frequency everywhere, every cycle budget matched to one wall-clock window

Failure mode	Repair
DDR5 CAS/RCD/RP timings were unsourced placeholders	Corrected to the DDR5-4800 speed bin, 40-39-39; full suite re-run
MLC multipliers were unsourced, attributed to a paper that does not support them, and the replacement read multiplier was itself a data error, read off the wrong column	Both derived from the EMBER macro's own published measurements, the read multiplier re-derived as 1.5x from its one- and two-bit-per-cell read bandwidths

Every repair was validated before its results were adopted, against a device-level anchor, exact predicted arithmetic or the simulator's own counters; the extended book's simulation-fidelity audit records each check. The fourteenth mode, the DDR5 supply and current parameters, is recorded there as ***partially*** *fixed: the supply voltage was corrected and the currents calibrated to vendor datasheets, but datasheet IDD values are specification limits rather than typicals, so what remains is a disclosed characteristic of the source data and not a repairable defect.*

3. What the pipeline taught me

Every one of the repairs in Table 3 began as a number that looked right. The lessons below are what I would tell the next person to build a cross-layer memory evaluation. They are stated as mechanisms, because a mechanism transfers and a retired value does not.

1. Match the array organization before you compare cells. I compared two ReRAM cell types for months at column-mux settings chosen independently for each. NVSim charges memristor cells no leakage at all, so a chip's leakage is peripheral circuitry and follows mat count: two different muxes meant two different mat counts, and what I was reading as a property of the access device was a property of my own configuration. The comparison became a device comparison only once the organization was forced identical and verified on every run.

2. Establish the time base and the window before you read any rate. My trace parser emitted timestamps without ever establishing what one unit of them meant, and the replay window I quoted followed from that unstated assumption; writes per second, endurance years and the admitted request population all inherited it. A rate is a count divided by a time, and if the time is unestablished the rate is decoration. Fix the unit first, and reconcile the trace against the generator's own counters.

3. The classic latency metric can lie under saturation. The in-controller average latency rises with queue depth while the time the workload actually waits falls: a short queue reports a fast average over the few requests it admits, and the rest of the burst backs up outside the controller where the total-latency counter cannot see it. Under a load that saturates the controller the honest measure is end to end, and both numbers belong in the same table.

4. A configuration template inherits more than it declares. The DDR5 baseline was derived from a DDR3-era example configuration. Raising the clock did not rescale the cycle counts that came with it, so refresh, write recovery, power-down and the activate window were all running at DDR3 cycle counts on a DDR5 clock, and nothing in the simulator objected. A cycle count is not a time. Every inherited key needs to be traced to a standard or a datasheet and labeled with which one, including the keys that look boring.

5. Read a simulator's power accounting to the line. The background-power path divided a rank's background power by its device count and never multiplied it back, so the reported module power was a per-device figure wearing a module label. No warning, no anomaly, a plausible number. Where a

simulator sums, averages or normalizes, read the source and anchor the result against an independent target.

6. Charge a device latency exactly once. The generator wrote the full NVSim device read latency into both tRCD and tCAS, so every read paid for the array twice. The repair was a rule and a check rather than a constant: tRCD plus tCAS must sum to the device read latency rounded up to whole interface cycles, and the generator re-reads the rendered configuration to confirm it. A device-to-protocol mapping needs that closed loop, because the error it catches is invisible in the output.

7. Keep a silicon counterweight beside every projection. A circuit-level model will happily project a nanosecond array, and no fabricated ReRAM part reads or writes in nanoseconds. Running two published-silicon configurations on the same traces cost very little and changed how every latency result here has to be read. A projection without a fabricated reference beside it is an argument about a model, not about a technology.

8. Know which timing key sits on the request path. In the default configuration the cell write time sits in write recovery, and reaches the request path only when a write closes its row, where the simulator charges it to the subarray's next activation; a write into an already open row charges nothing beyond the burst and the turnaround, and neither branch delays the write's own completion. Moving the same count onto the write pulse charges every write instead. Which key holds the number decides whether a write penalty reaches the results at all.

4. Results

4.1. Latency

Table 4: Average total latency (ns), eight full-DIMM configurations against the six traces, 250 ms matched window. The projected rows are NVSim media timings under the three-part caveat of Section 1.3.

Technology	GCC	LBM	STREAM	GPT-2	AlexNet IFMAP	AlexNet OFMAP
DDR5-4800	83.1	78.3	79.4	222.1	212.3	234.3
1T1R SLC	41.4	43.1	42.3	143.3	143.6	353.1
1S1R SLC	36.8	38.1	37.3	130.1	130.0	391.5
1T1R MLC	44.5	42.9	46.6	157.2	154.1	512.0
1S1R MLC	37.9	35.5	36.9	129.6	131.4	652.9
PCM (legacy)	12,168.2	5,136.8	5,190.7	1,835.8	1,871.3	4,747.1
1T1R SILICON	11,888.5	17,349.5	27,802.3	10,934.1	10,758.9	80,066.6
1S1R SILICON	378,763.1	324,174.4	530,595.6	193,995.8	193,699.1	1,092,759.4

Average total latency, wall-clock nanoseconds: queue entry to completion, the like-for-like device comparison. The four ReRAM rows above PCM are NVSim-projected device timings at the matched 2048 x 2048 organization; the last two rows replay the identical traces at published fabricated-chip timings. DDR5 and all four projected tracks complete every workload in full; PCM, 1T1R SILICON and 1S1R SILICON complete as little as 46.1%, 9.2% and 0.5%, and a latency average over a service-limited run

covers only that run's completed prefix, which is why end-to-end latency is the saturation measure. Source: results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv, column Latency_ns.

On the three CPU traces the projected ReRAM DIMM has lower average memory latency than the DDR5-4800 baseline, and under GCC the margin is about two to one: 41.4 ns for 1T1R SLC and 36.8 ns for 1S1R SLC against 83.1 ns (Table 4). The baseline runs NVMain's open-page policy while the ReRAM configurations run its relaxed close-page policy; re-run under the same close-page policy the baseline gives 78.2 ns under GCC, 69.9 ns under LBM and 93.5 ns under STREAM, so at matched policy the GCC margin is 1.9 to one for 1T1R SLC and 2.1 to one for 1S1R SLC, and on STREAM, where the open-page baseline serves 45% of requests from the open row, close-page makes the baseline slower. The baseline carries its JEDEC timings, write path and refresh blocking (Section 2.3), while the ReRAM subarray reads in 10.12 ns (1T1R) or 4.70 ns (1S1R) and precharges in one 1.25 ns cycle. The 0.797 W DDR5 power figure that Table 5 retains comes from the open-page run; the close-page runs are used only for the latency margin.

The decomposition of that margin belongs before any interpretation of it. Of the 41.40 ns 1T1R SLC average, **11.25 ns is NVSim device time**, charged once as tRCD plus tCAS; the other 30.2 ns is NVMain's default protocol cycle counts at the assumed 800 MHz interface that Section 2.3 records as carrying no ReRAM citation. Roughly three quarters of the ReRAM figure is therefore an interface assumption rather than a device measurement, as is the DDR5 figure in the other direction, whose protocol timings are JEDEC's. **The direction of the comparison is robust**: ReRAM stays below DDR5 at all three interface clocks measured here. **Its magnitude is model-dependent**, and 41.4 against 83.1 ns is the outcome of one clock choice rather than a device ratio; Figure 2 shows what the choice is worth.

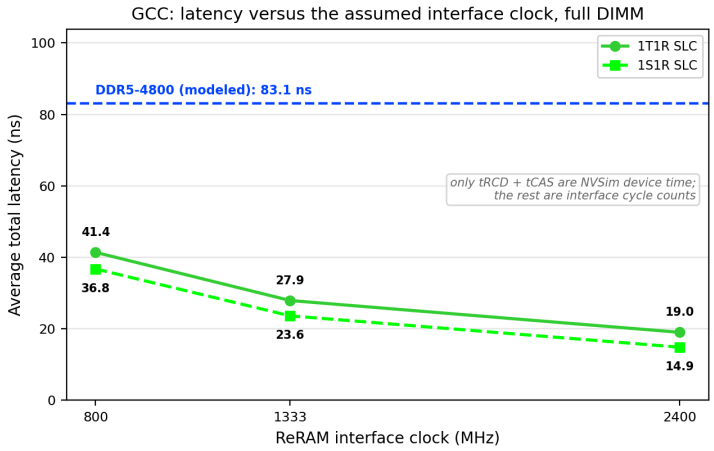


Figure 2: GCC latency against the assumed ReRAM interface clock. 1T1R SLC falls 41.4, 27.9, 19.0 ns and 1S1R SLC 36.8, 23.6, 14.9 ns at 800, 1333 and 2400 MHz, against the modeled DDR5 baseline of 83.1 ns. Only tRCD plus tCAS are NVSim device time; every other protocol timing is a constant in interface cycles and therefore scales with the clock.

The write-dominated burst reverses the verdict. On the AlexNet output map DDR5 averages 234.3 ns against 353.1 ns for 1T1R SLC, 391.5 for 1S1R SLC, 512.0 for 1T1R MLC and 652.9 for 1S1R MLC, so DDR5 is 1.507x and 1.671x faster than the two SLC tracks and 2.185x and 2.787x faster than the two MLC tracks. Those ratios are a *lower* bound on the deficit, because in the default configuration the cell write pulse does not delay the write's own completion at all.

MLC, and where the write pulse is charged

MLC is barely slower than SLC in Table 4, and that is a consequence of this project's device-to-protocol mapping rather than a device finding. The generator books the cell write latency as tWR and leaves the write-pulse parameter tWP at zero, so, as lesson 8 describes, it reaches the request path only when a write closes its row and never delays the write's own completion, 13.75 ns on every track; the 49.79 ns that separates a 1T1R MLC write from a 15.26 ns SLC write never reaches the write.

Rather than assert the size of that bound, the primary matrix was re-run with the choice inverted: the same latency written to tWP, tWR held at one cycle, so every write's completion carries the pulse and an open-row write charges it to its subarray's next read and next write. Everything else is identical, and the two all-read traces come back byte-identical between the datasets. **The write-pulse bracket table** below sets the two ends side by side.

The write-pulse bracket table: average total latency (ns), full-DIMM, default mapping against the bounding run.

Track	GCC, default	GCC, bounding	OFMAP, default	OFMAP, bounding
1T1R SLC	41.4	48.3	353.1	696.6
1T1R MLC	44.5	65.8	512.0	1,649.9
1S1R SLC	36.8	47.4	391.5	916.6
1S1R MLC	37.9	72.4	652.9	2,498.7

Default values are the Latency_ns column of results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv; bounding values are the same column of results/system_rev2026-09_writepulse/_csv/processed_bar_chart_metrics.csv, in both cases the full_dimm rows. The two all-read traces are omitted because they are identical in the two datasets. These are media latencies at an assumed 800 MHz interface, not device latencies of any shipped part.

Read against DDR5's 234.3 ns on the output map, each ratio above becomes a bracket, derived from the two OFMAP columns of the write-pulse bracket table: 1.507x to 2.973x against 1T1R SLC, 1.671x to 3.912x against 1S1R SLC, 2.185x to 7.042x against 1T1R MLC and 2.787x to 10.665x against 1S1R MLC. MLC costs up to 1.53x SLC on CPU traffic and up to 2.73x on the write burst, derived the same way.

What produced the bracket is not the same thing on the CPU traces as on the write burst. On GCC, LBM and STREAM the rise equals the write fraction of the completed requests times the cell pulse to within 1% on every track-trace pair, which is what the simulator's own row-buffer counters predict, because those traces have almost no row hits at all: the hit share is about 0.03% under GCC and effectively zero under LBM and STREAM, so nearly every write closed its row and the default mapping had already charged the cell to the subarray's next activation. On the AlexNet output map the picture inverts. That trace is pure writes, 76% of its accesses hit an open row, and the measured rise is 21 to 23 times the per-write pulse, so the headline write-burst bracket is mostly subarray blocking that the default mode never charged rather than the write's own completion latency. **The bracket is an upper end of the write-path cost inside this model, not a prediction of a device**, and a reader who takes the output-map figure as the cell pulse added to every write will be wrong by a factor of about 22. Two consequences follow. "Below DDR5 on the CPU traces" survives for all four projected tracks at the upper end, the worst case in the matrix being 1S1R MLC under GCC at 72.398 ns against DDR5's 83.111 ns, which is 0.871x

of DDR5 (derived); and no endurance number moves, because the same traces present the same writes and the mode changes only when they are served.

One comparison bounds the whole MLC treatment. Published program-and-verify data on 1T1R ReRAM reports an average of 49 pulses per level move, median 36 **[28]**, and the closest published two-bit *array* result is MLC phase-change memory, more than seventy iterations for a total write of 9.7 us **[29]**; against that, the 3.263x write multiplier used here understates the media-level MLC write cost by one to two orders of magnitude, and the bounding run does not close that gap, because it moves this project's own pulse onto the request path rather than replacing it with a verify loop. On the read side the honest statement is an absence: no primary source pairing SLC and MLC array-level read times was found. TLC and QLC carry no defensible number at all, only a qualitative trend, that verify iterations grow and the sensing window and retention worsen as levels are added.

4.2. Power

NVSim reports no transistor-versus-selector leakage gap. At the matched organization it assigns both cell types the same 108.384 mW of peripheral leakage per 1 Gb chip, and all four ReRAM full-DIMM configurations sit on one shared static floor of 6.937 W. That is a statement about the periphery: NVSim models no selector leakage of its own, and the 1S1R characterization is uncalibrated against any fabricated cross-point part, so the selector's contribution is bounded separately below. Table 5 is the module accounting.

Table 5: total module power (W) and power per gigabyte (W/GiB), full-DIMM, module-sum semantics.

Technology	GiB	GCC (W)	Suite range (W)	GCC (W/GiB)
DDR5-4800	16	0.797	0.797 to 1.110	0.0498
PCM (legacy)	4	0.046	0.036 to 0.046	0.0115
1T1R SLC	8	6.939	6.939 to 7.291	0.8674
1S1R SLC	8	6.940	6.940 to 7.318	0.8674
1T1R MLC	16	6.941	6.941 to 7.287	0.4338
1S1R MLC	16	6.941	6.941 to 7.353	0.4338

Whole-module sums; the suite range is the lowest and highest of that row's six workload values, and the per-GiB column divides each module total by its own capacity, which is the only form in which these technologies are comparable. ***Every figure in this table counts memory devices only*** *- no register clock driver, power-management IC, PHY, termination or ReRAM-side controller, on either side. ReRAM's power-down mechanism is mechanically live but its power-down energy is an explicit, disclosed placeholder equal to existing standby energy, so no ReRAM figure claims a gating benefit and every per-GiB ratio here is an upper bound on the gap a gated ReRAM module would show. Source: results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv.*

Read per gigabyte, **ReRAM SLC costs 12.7x to 17.4x DDR5's power and ReRAM MLC 6.4x to 8.7x** across the suite, MLC halving the penalty because leakage is per chip and MLC puts twice the capacity behind the same chips. The legacy PCM baseline sits *below* DDR5 per gigabyte, which reflects the leakage constant of the configuration it inherits rather than a characterized device. Figure 3 shows where the watts are: dynamic power is 2.9 to 4.9 mW module-wide under GCC and rises only to 346 to 417 mW under the AI read bursts, under 6% of the total. DDR5′s refresh alone, 9.1% of its 0.797 W under GCC or

about 73 mW, is fifteen to twenty-five times the whole ReRAM module's dynamic draw, and ReRAM refreshes nothing; the 6.94 W leakage floor outweighs that saved refresh ninety-six times over. ReRAM power is leakage, and at the matched organization that is a statement about both cell types equally.

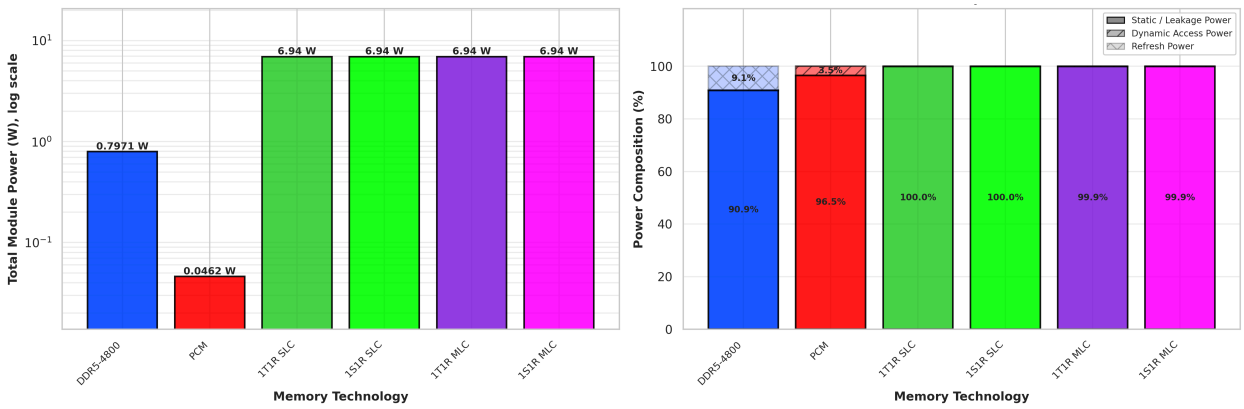


Figure 3: system power breakdown under GCC. All four ReRAM tracks sit on one shared leakage floor near 6.94 W for 8 GiB (SLC) or 16 GiB (MLC), against a 16 GiB DDR5 module. The chart is a module-level view; the per-gigabyte normalization, which is the only form in which these modules are comparable, exists only in Table 5.

That deficit is not the end of the arithmetic, because a non-volatile module can in principle gate the periphery a DRAM module cannot. Gating the static component for an idle fraction f leaves $(1 - f) \times 6.937 + 0.003$ W, so reaching DDR5's 0.399 W for the same 8 GiB requires $f = 94.3\%$. **That threshold sits inside, not above, the idleness band this project cites**: the 2 to 6% memory-bandwidth utilization reported for web-search and data-analytics servers **[30]** is 94 to 98% idle, and the band brackets break-even, 0.142 W against DDR5's 0.399 W at the idle end and 0.419 against 0.399 W at the busy end. What the model cannot price is the gated module itself, because no citable ReRAM power-gating characterization exists; and fixed per-module overhead moves the arithmetic against ReRAM, since 8 GiB of ReRAM carries the same register, PMIC and PHY overhead O as 16 GiB of DDR5, so the required gating fraction *rises* with O, 94.3% at $O = 0$, **97.9% at $O = 0.5$ W per module**, and beyond **$O = 0.79$ W** parity is out of reach at any gating fraction. The same overhead compresses the ungated ratio and tightens the gated comparison, and the parity claim rests on the second.

Two device-level results complete the picture. **The leakage figure is an organization result, not a cell result.** At a 1024 x 1024 organization both cells leak about twice as much, 224.138 and 220.224 mW per chip, still within 2% of each other, so whatever organization is matched the two cells land together. **The selector's sneak current adds nothing to standby power and little during an access.** The standby adder is exactly zero at both selector bounds; during an access the budget is 7.56 mW per chip at an ovonic-threshold-switch bound and 0.076 mW at a best-published FAST bound, which at the OTS figure is 0.87% of the module's 6.937 W static floor while a rank is being accessed. What binds is tile validity, not leakage: the OTS bound supports a tile of at most 1666 cells a side, so **the 2048 x 2048 organization simulated here is not valid at present-day selector quality**, while the FAST bound supports 166,666. Every 1S1R number in this book is conditional on a selector better than today's production ovonic threshold switch. *Source: results/selector_layer.json.*

4.3. Endurance

Writes wear the cell, so endurance is the constraint that decides whether any of the above matters. The projection carries the write distribution measured per line location inside the 250 ms window onto a 64 GiB module at the best-sourced 10^6 cycle rating **[31]**, and asks how long it lives under four wear-leveling policies. Table 6 is that projection for the three sustained workloads, and Figure 4 is the same projection by policy.

Table 6: projected lifetime and required endurance (1T1R SLC full DIMM, 64 GiB, 10^6 cycles/cell, 250 ms window, one write per line per request).

Workload	Write rate	No leveling	Rand. Start-Gap	Ideal	Req. endur. (ideal)
GCC	0.88 M/s	7.7 h	7.7 to 15.4 h	38.6 yr	2.6×10^5
LBM	9.54 M/s	23.2 h	0.20 yr	3.6 yr	2.8×10^6
STREAM	7.99 M/s	69.5 h	0.96 yr	4.3 yr	2.3×10^6

Ranges in the randomized Start-Gap column mark cases where the hottest line fails before the gap completes its first rotation, so the true lifetime lies between the no-leveling value and twice it. ***Required endurance*** *is the per-cell cycle count a ten-year life would demand at this write rate and capacity, which is the same equation solved for endurance rather than for time.* ***Every row assumes a stationary write distribution:*** *the spatial pattern of writes measured inside the 250 ms window is taken to repeat unchanged for the module's whole life. Source: results/endurance_table.csv, rows capacity_gib = 64, endurance = 1e6, write_reduction = 1.0, rate_basis = admitted.*

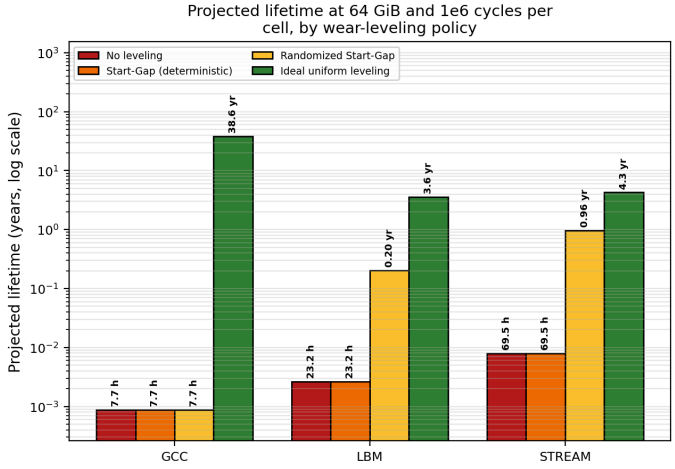


Figure 4: projected lifetime by wear-leveling policy. Every value is a projection from the write distribution measured in a 250 ms window, assumed stationary for the module's life; never a measured lifetime.

The distance between those columns is the finding. **Endurance viability is decided almost entirely by the wear-leveling mechanism, not by the cell.** With no leveling the module survives *hours* under every sustained workload, because the traces are sparse: GCC touches 577 row locations, LBM 826 and STREAM 1,954 out of the 131,072 rows of the simulated 8 GiB module, and of 1,048,576 at the 64 GiB capacity Table 6 projects. A workload that concentrates its write stream on a few hundred locations

exhausts them however much unused capacity sits beside them. With ideal leveling the same module reaches 38.6 years under GCC and 3.6 under LBM, four to five orders of magnitude away.

Deterministic Start-Gap **[32]** does not close that gap for these footprints. Over a single whole-module region the hottest line fails before the gap has rotated once, so it recovers essentially nothing; randomizing the region assignment recovers part of it in the two streaming cases (0.20 years under LBM and 0.96 under STREAM against ideals of 3.6 and 4.3) and still nothing under GCC, whose footprint is too small to randomize over. One consequence runs against intuition: over a single region **a larger module is worse, not better**, because the rotation period grows with the region while the gap still advances one line per hundred writes: projected under randomized Start-Gap, GCC survives 0.69 years at 8 GiB but only 7.7 hours at 64 GiB.

Solved for endurance instead of for time, a ten-year life on a 64 GiB module demands **2.6×10^5 cycles under GCC, 2.8×10^6 under LBM and 2.3×10^6 under STREAM** with ideal leveling, and 6.3, 6.5 and 4.3×10^6 under randomized Start-Gap: against the 10^6 planning value GCC clears the bar fourfold under ideal leveling and everything else fails it by 2.3 to 6.5x. A ten-year ReRAM main memory needs a cell in the high 10^6 range *and* a leveling scheme better than a single-region Start-Gap.

One negative result bounds what the simulation can contribute. **It cannot show wear leveling at work.** With one Start-Gap region of 67,108,864 lines (4 GiB: NVMain sizes the region from the per-channel geometry, without the channel count, and the two channels share one gap register) and one gap move per hundred writes, a 250 ms window makes 2,207 gap moves under GCC and 23,848 under LBM, roughly 3×10^{-5} and 4×10^{-4} of one rotation, so running the matrix with the remapper off reproduces the primary run's wear statistics and latencies identically. At those move rates one rotation of the region takes about 7,600 s of simulated time under GCC, 2.1 hours, and about 700 s under LBM, 11.7 minutes: 30,000 and 2,800 windows of 250 ms, far beyond any trace this pipeline could replay. Leveling's effect is carried entirely by the projection behind Table 6, validated against a write-by-write simulator to within 0.3%.

4.4. Scaling and sensitivities

Seven sensitivity series each change one flag against the primary matrix, and a pre-revision NVSim voltage sweep sits beside them. Table 7 takes the chip-count axis at the two extremes of behavior.

Table 7: chip-count scaling, 1T1R SLC, average total latency (ns) and Power-Delay Product (W·ns), 250 ms matched window.

Workload	Lat. 8	Lat. 16	Lat. 64	PDP 8	PDP 16	PDP 64
GCC	45.4	44.8	41.4	39.5	77.8	287.3
AlexNet OFMAP	391.3	352.1	353.1	375.0	657.6	2,496.2

Chip counts are 8, 16 and 64 devices; the one-chip point of the series is omitted because its decoded address space is a generator floor rather than a physical module. Source: results/ rev2026-09_primary_csv/processed_bar_chart_metrics.csv, rows Technology = 1T1R_SLC.

The latency return on scaling is a queueing return, not a parallelism return. GCC moves only from 45.4 ns at eight chips to 41.4 at sixty-four, and the write burst is flatter still, 352.1 against 353.1 ns between sixteen chips and sixty-four, for a 3.8-fold efficiency loss. The two AI read bursts do gain, by 1.68x, and a one-channel control run shows the gain is entirely the second channel rather than the added

devices; rank depth, isolated the same way, buys at most 7.5% on any of the six traces. Figure 5 plots the same trajectory in the power-latency plane.

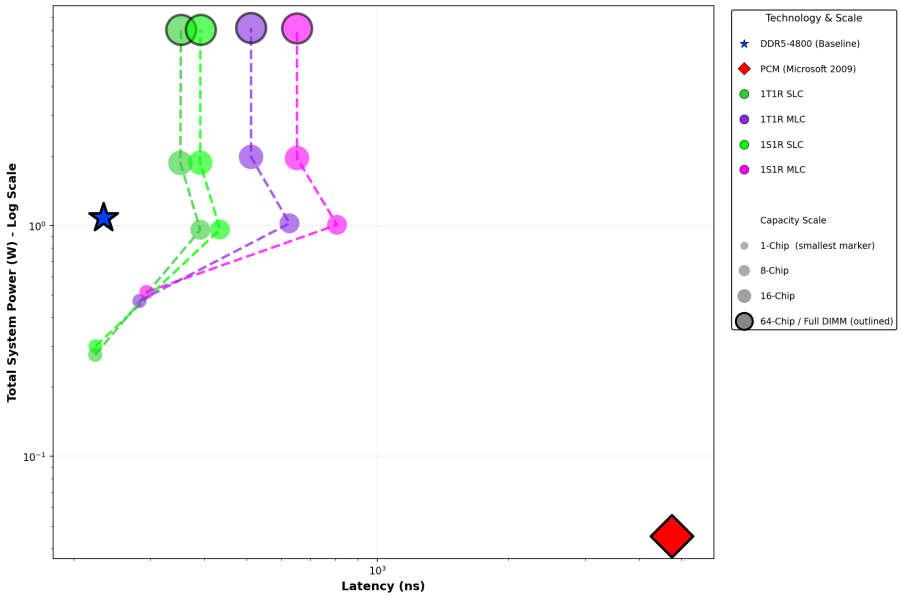


Figure 5: Pareto scaling trajectory under write-torture conditions (AlexNet OFMAP), all four ReRAM tracks with the two baselines. The leftmost point of each trajectory is the single-chip architecture, a non-physical generator floor that enters no conclusion here.

Queue depth changes how the results should be read rather than what they say. Across depths of 8, 32 and 128 the CPU traces do not move at all, but under the AlexNet output-map write burst the two latency measures move in opposite directions: for 1T1R SLC the in-controller average goes 124.9, 353.1 and 1,049.2 ns while the end-to-end average goes 680.1, 486.1 and 432.5 us. A deeper queue *raises* the classic metric and *lowers* the time the workload actually waits, because a short queue reports a fast average over the few requests it admits while the rest of the burst backs up outside the controller where the total-latency counter cannot see it. That answers the write-queue sizing question of Section 2.1, and it is a *write* effect: on a write burst a deeper queue pays off end to end, and the in-controller average reports that gain as a regression.

Three further series, one sentence each. At a 1024 x 1024 organization the 1T1R SLC full DIMM is 4 to 10% faster and every track pays 14.1 to 14.7 W against 6.9 W, the two cells still leaking within 2% of each other (Section 4.2). A 64 B DDR5 cross-check on a legacy single-channel shape averages 79.6 against 83.1 ns under GCC but holds tCCD below the JEDEC floor, so its advantage is optimistic. And a pre-revision three-point NVSim ReadVoltage sweep, plus or minus 20%, not re-run at the matched organization or under the repaired power model, moved the power-delay product by less than 4% with read latency invariant; its numbers are not comparable with Tables 4 to 7.

4.5. Density, and the comparison in one table

Density is the axis on which the selector earns its place. At the matched organization NVSim gives the 1S1R chip a die area of 3.540 mm² against 12.008 for 1T1R, a 3.4x advantage that is now the selector's only device-level advantage, and it converts into a die-level density of **1.24x DDR5 for 1S1R SLC and 2.47x for 1S1R MLC** against **0.36x and 0.73x for the two 1T1R tracks** (Figure 6): the transistor-gated

cell is less dense than the DRAM it would replace, and only the cross-point clears parity. Scaled quadratically to a 12nm-class node the same anchors project 4.15x and 8.31x for 1S1R SLC and MLC and 1.22x and 2.45x for 1T1R, with deck stacking available only to the cross-point. Those columns are geometry, not characterization, and are linear in an assumed cell area: a realized 1S1R cell at twice the idealized $4F^2$ would drop 22nm SLC below parity.

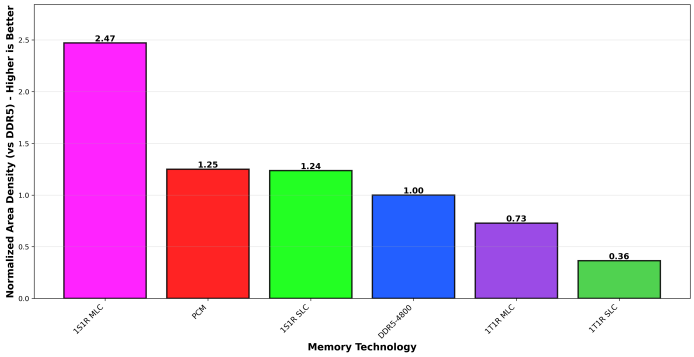


*Figure 6: die-level density normalized to DDR5 = 1.00, higher is better. The PCM bar, 1.25x, is **not characterized**: a fixed constant in the metrics pipeline inverted from an older 0.80 mm²/GB placeholder, with no device or datasheet source behind it.*

Table 8 puts every axis side by side.

Table 8: cross-technology summary, full-DIMM configurations.

Technology	GCC lat. (ns)	GCC power (W / W per GiB)	Geo-mean PDP	Die density	Lifetime @128 GiB
DDR5-4800	83.1	0.797 / 0.0498	136.2	1.00	n/a (volatile)
PCM (legacy)	12,168.2	0.046 / 0.0115	173.6	1.25*	not evaluated
1T1R SLC	41.4	6.939 / 0.8674	641.0	0.36	7.1 yr
1S1R SLC	36.8	6.940 / 0.8674	594.5	1.24	7.1 yr
1T1R MLC	44.5	6.941 / 0.4338	722.8	0.73	7.1 yr
1S1R MLC	37.9	6.941 / 0.4338	645.4	2.47	7.1 yr

*Power is module watts / watts per gigabyte; only the second is comparable, because the modules are 16 GiB (DDR5, ReRAM MLC), 8 GiB (ReRAM SLC) and 4 GiB (PCM), and both sides count memory devices only. Geo-mean PDP is W·ns over the six workloads. ***** The PCM density figure is a fixed constant in the metrics pipeline with no device or datasheet source behind it, not a characterized die area; it is reported here for completeness and should be read as* not characterized, *like the lifetime cell beside it. The published-silicon counterweights of Table 4 are deliberately not repeated here: no row of this table describes a fabricated part. **Projected lifetime** is the LBM worst case at 128 GiB under* ideal *uniform leveling at 10^6 cycles per cell, and is a projection in the sense of Section 4.3, not a measured lifetime. Source: results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv and processed_geometric_means.csv.*

5. Discussion

5.1. The verdict

Chapter 4 produces three kinds of statement, and keeping them apart is the discipline of this book.

What holds is direction. On the CPU traces the projected media is faster than the DDR5-4800 baseline, and that ordering survives at the upper end of the write-pulse bracket; at a matched organization the cross-point clears DDR5's die-level density and the transistor-gated cell never does (Section 4.5). Neither magnitude is robust in the way its direction is.

What is bounded is the power deficit and the write burst. The per-gigabyte penalty counts memory devices only on both sides and is measured against a baseline that realizes its JEDEC power-down credit while ReRAM's is a no-savings placeholder; gating break-even sits inside, not above, the cited idleness band **[30]**, so the arithmetic brackets parity for that class and cannot price it. The upper end of the output-map bracket is an upper end of the write-path cost inside this model, not a prediction of a device.

What is projected is every lifetime, and the latency advantage itself. The endurance claim lives in the leveling mechanism, not the cell, and no figure here is a measured lifetime; the latency result is a projection in the strict sense of the caveat of Section 1.3, and the same DIMM at published silicon timings runs in microseconds.

5.2. Near and far memory

Everything above places the module in the *near* memory position, reached over the processor's own latency-controlled interconnect, where latency is the figure of merit. A second position, reached over PCI Express instead, values capacity and density above the last fifty nanoseconds; it adds memory from a slot that would otherwise carry none, not bandwidth beyond what a socket's own channels already deliver. This section asks where the projected device would sit if it were aimed there, and answers with measured numbers.

The window real systems accept is narrow and it is measured. Local DRAM at idle sits well below it: DDR5 measures 111 to 114 ns on two-socket servers, with platform baselines of 78 and 115 ns in Microsoft's Pond study, and the far socket of the same machine costs 140 to 193 ns **[33]**, **[34]**. Shipped CXL-attached DRAM expanders measure about 214, 239 and 271 ns idle load-to-use for ASIC-based devices and about 394 ns for an FPGA-based one **[33]**, **[35]**. What does *not* survive contact with that data is a single blanket threshold for acceptable far memory: Pond's workload study finds applications losing measurable performance at latency increases well below the width of that band while most tolerate an increase of that size **[34]**, so acceptability is a per-workload property, not a number. And **the few-hundred-nanosecond band belongs to CXL-attached DRAM *devices*, not to local DRAM**, so no figure drawn from it is a DRAM reference.

This book's two latency lines sit on opposite sides of that window, and neither is inside it. The projected media is 36.8 to 44.5 ns under GCC and 353.1 to 652.9 ns on the write burst, so on CPU traffic it is below the entire window, with room for a controller and an error-correction stack before it would reach the low end. The published-silicon counterweights, 11.9 us and 379 us under GCC, are far above it. That gap is the state of the technology: nothing shipped is close, and nothing in the device physics says it could not be.

One mechanism decides how such a device would be used. A CXL-attached region surfaces to Linux as a CPU-less NUMA node, with the ACPI Heterogeneous Memory Attribute Table supplying latency and

bandwidth per initiator-target pair so the kernel can rank it **[36]**, and what the kernel does with that ranking is **page placement, not per-request routing**: pages move between tiers at 4 KiB granularity on a millisecond-to-second timescale, and nothing routes an individual load or store by projected access time. A far memory can afford to be slower than DRAM; it cannot afford to be slower by a factor page migration cannot amortize.

The bandwidth half of the same claim does not hold as stated either. A shipping x8 CXL 2.0 expander is published at up to 36 GB/s **[37]**, at or below a single DDR5-4800 channel's 38.4 GB/s **[35]** and far below the eight such channels of a socket, so **CXL adds capacity and bandwidth from a slot that otherwise carries no memory at all**, not bandwidth a DIMM channel could not deliver. A shipped controller sets the shape of the class: a CXL 1.1 / 2.0 smart memory controller carries two DDR5 channels and up to 2 TB behind one slot, and publishes no latency figure at all **[38]**. That framing is what makes the density dividend of Section 4.5 the quantity such a slot is bought for.

Table 9 is the one place in this book where the modeled tracks sit beside measured hardware, each row labelled with the kind of number it carries.

Table 9: device reference row, this book's modeled media beside measured devices.

Row	Kind	Latency	Note	Source
1T1R SLC, full DIMM	media	GCC 41.4 to 48.3 ns; burst 353.1 to 696.6 ns	smallest write penalty of the four	primary CSV and the write-pulse bracket table
1T1R MLC, full DIMM	media	GCC 44.5 to 65.8 ns; burst 512.0 to 1,649.9 ns	twice the capacity per chip	as above
1S1R SLC, full DIMM	media	GCC 36.8 to 47.4 ns; burst 391.5 to 916.6 ns	conditional on a selector better than production OTS	as above
1S1R MLC, full DIMM	media	GCC 37.9 to 72.4 ns; burst 652.9 to 2,498.7 ns	densest track, 2.47x DDR5	as above
DDR5-4800 baseline	media (modeled)	GCC 83.1 ns; burst 234.3 ns	two 32-bit subchannels, 64 B per access, JEDEC timings	primary CSV
DDR5, two-socket server	device	111 to 114 ns idle; 140 to 193 ns to the far socket	measured on shipped platforms	**[33]**, **[34]**
CXL-attached DRAM expanders	device	214, 239, 271 ns (ASIC), about 394 ns (FPGA), idle load-to-use	shipped expander modules behind a link	**[33]**, **[35]**
Optane PMem 100 / 200	device	305 ns idle random read; 169 ns sequential read	6.6 GB/s read, 2.3 GB/s write per module; no open loaded-latency figure	**[19]**

The four ReRAM rows share the assumed 800 MHz interface and carry no controller, ECC or PHY. ReRAM and DDR5 rows are the Latency_ns column, full_dimm, of results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv and, for the second figure of each range, of results/system_rev2026-09_writepulse/_csv/processed_bar_chart_metrics.csv; "burst" is the AlexNet output map. DDR5 appears twice on purpose: as the modeled media number this book compares against, and as the measured device latency of a real server, about a third higher. ***No ReRAM row here is a device number, and no device row is a prediction for ReRAM.***

The conditional statement that follows is honest in both directions. **ReRAM could serve as far memory only if a *device* built on it, controller, error correction, and a DRAM cache in front of it as in**

Optane's Memory Mode [39], landed in the same few-hundred-nanosecond class as the measured CXL-attached DRAM devices above. No published ReRAM silicon does. What the projected numbers do say is which half of that problem is open: at 36.8 to 44.5 ns of media latency under CPU traffic, with 1.24x to 2.47x DDR5's die-level density on the selector track, the device physics is not the obstacle. The interface, the error-correction stack and the controller are.

5.3. Reliability: bit errors and error correction

The selector layer of Section 4.2 answers whether a 2048-cell tile can be *read* at all, not how often the bit that comes back is wrong or what correcting it costs. Neither NVSim nor NVMain models a bit error anywhere in this pipeline, so what follows is a literature result set, and three categories are kept apart throughout, because literature that conflates them is unusable here: *retention* errors, a cell drifting out of its window while idle; *endurance* errors, a cell cycled past its rating, which hard-fails; and *read-disturb* errors, a cell perturbed by the reads of its neighbours or of itself.

Layer one, the raw device rate. For the cross-point track **no measured array bit error rate is published at all**. The only numeric figure is a channel model: a 1024 x 1024 array at 10 ohm per junction with an *ideal* selector, which gives a read bit error rate from 4.29×10^{-4} for the best-positioned cell to 7.33×10^{-2} for the worst **[40]**. The ideal-selector assumption is exactly what Section 4.2 exists to question, so that figure is a model output and never silicon. The 1T1R track does have measured macros, and they span orders of magnitude with the write scheme: a 128 kb macro with a smart-write algorithm reports under 10^{-7} after 10^{6} cycles **[41]**, while HfO_2 arrays under adverse cycling sit between 10^{-3} and 10^{-2} **[42]**. The usable statement is a band, and where a design lands inside it is a write-algorithm and margin decision rather than a material property.

Layer two, what the selector itself contributes. A measured 1S1R selector stacked with STT-MRAM reports a per-read disturb rate of 10^{-9} and 10^{9} read-disturb-free cycles **[43]**, and an ovonic-threshold-switch cross-point cell reports a failure rate of 9% falling to 0.02% after read-endurance optimization **[44]**. **The selector's own contribution is therefore bounded well below the layer-one array rates**, which is the result that matters here: the 1S1R disadvantage established in Chapter 4 is tile validity and sneak current, not error rate.

Layer three, the design point an NVM main memory actually uses. The standard design point in the persistent-memory architecture literature is a raw rate of 2×10^{-4} corrected with BCH(2312,2048,22), at about 27% coding overhead **[45]**; DRAM's on-die SECDED, by contrast, tolerates only about 10^{-6} raw **[46]**, and the only shipped NVM media has an SSD sibling rated by datasheet at under one uncorrectable sector per 10^{17} bits **[47]**. Side by side the three layers give the answer this section exists for. **A main memory built on published raw ReRAM error rates is possible, but it needs SSD-class rather than DRAM-class correction, at roughly a 27% cost in bits.** The stronger claim sometimes made in discussion, that the error rate rules out a memory system altogether, is not supportable against this literature.

Two closing statements belong here. The selector layer of Section 4.2 was built independently of this literature and agrees with it: it applies the same **10% minimum read-margin criterion [48]**, and the limit that binds the 2048-cell tile is the line-resistance and tile-leakage mechanism, not a sneak-path bit error rate. And, plainly: **the modeled latencies exclude ECC decode.** The generated configurations carry no coding block and no decoder, so every latency in Chapter 4 is optimistic by exactly the decode time a 27%-overhead code would add, on top of being a media rather than a device number.

5.4. Limits of the model

Seven limits bound every number in this book, and all seven are disclosed rather than discovered.

The 800 MHz interface is assumed: it carries no ReRAM citation, the only datasheet-verified ReRAM chip I/O clocks located run 80 to 160x slower, and roughly three quarters of every ReRAM latency average is protocol cycles at that clock (Sections 2.3, 4.1). **The write path is a bracket, not a model**: the cell write time is charged either as write recovery or, in the bounding run, to every write's completion, and neither end models the program-and-verify loop an MLC part actually runs (Section 4.1). **Power is device-only**: both sides count memory devices, with no register clock driver, power-management IC, PHY, termination or ReRAM-side controller, and the ReRAM power-down energy is a no-savings placeholder (Section 4.2). **No ECC decode sits on the read path**, as Section 5.3 states, and no coding overhead is charged against the density figures. **Endurance rests on stationarity**: every lifetime extrapolates a write distribution measured inside a 250 ms window to years, and the leveling schemes are modeled analytically rather than simulated end to end (Section 4.3). **Every ReRAM timing is projected**: no fabricated part has matched NVSim's array core, which is why the published-silicon counterweight accompanies every latency table. **There is no CXL, controller or device model anywhere in this book**: Section 5.2 positions the projected media against measured devices; it does not simulate one.

6. Conclusion and Future Work

At 22nm, on a matched 2048 x 2048 organization and six workloads, the projected ReRAM DIMM is faster than a DDR5-4800 baseline on CPU traffic and slower on a sustained write burst, denser than DDR5 as a selector-gated cross-point and less dense as a transistor-gated cell, and more expensive per gigabyte in power by 12.7x to 17.4x as SLC, device for device and as an upper bound. Write endurance is the binding constraint and the wear leveling, not the cell, decides it. Every latency here is a media number for a projected array that no fabricated part has matched: the same DIMM built from published silicon timings averages 11.9 us and 379 us under GCC. *Neither cell is a drop-in DDR5 replacement at 22nm*; the selector-gated cross-point is a density play with competitive projected latency and a large, honest power deficit per gigabyte, and the transistor-gated variant matches it on leakage and loses on area.

What would overturn this verdict. Each negative result above carries the threshold that would reverse it, and stating them together is the most useful thing a negative result can do. The power deficit closes only if idle gating removes at least 94.3% of the static component with no module overhead, and no gating fraction closes it once the ReRAM module's own register, PMIC and PHY overhead exceeds 0.79 W (Section 4.2). The 1S1R results hold at the simulated 2048 x 2048 tile only with a selector that supports that tile side, which a production ovonic threshold switch, at 1666 cells, does not and a FAST-class selector does (Section 4.2). A ten-year life at 64 GiB needs a cell rated for 2.6×10^5 cycles under GCC and 2.8×10^6 under LBM with ideal leveling, or 4.3 to 6.5×10^6 under randomized Start-Gap (Section 4.3). The cross-point's density lead holds only while the realized cell stays near the idealized $4F^2$; at twice that area 22nm SLC falls below DDR5 parity (Section 4.5). And the latency advantage becomes a statement about hardware only when a ReRAM array and interface at the assumed 800 MHz exist: at published silicon timings the same DIMM averages 11.9 us and 379 us under GCC (Section 4.1).

Eight pieces of work follow directly from what this book could not settle.

- **Native MLC logic with the write pulse and verify on the path.** Replace the analytic MLC penalty with native circuit-level characterization of iterative sensing in NVSim, and put an iteration-count

distribution and a verify-read cost on the request path instead of one scalar pulse. That is the only way the MLC latency question stops being a modeling-choice question.

- **Faster-PHY interface parity.** The 800 MHz interface is a modeling choice, not a property of the media; the clock sweep of Section 4.1 quantifies it, and what remains open is a defensible ReRAM PHY rather than an assumed one.
- **The Optane reference row completed with loaded-latency data.** Table 9 carries the only shipped NVM main memory at its idle measured latency; no open figure exists for it under load, and that gap is what keeps the row from being a full comparison.
- **A CXL Type 3 device model with capacity-bound traces.** Model the far position directly: a memory region behind a link model, exposed as a CPU-less NUMA node, driven by traces that are *capacity*-bound rather than latency-bound, which none of the six traces here is. The deliverable is the far-memory counterpart of Table 9, with this project's own rows measured rather than positioned.
- **ECC decode on the read path.** Section 5.3 leaves the largest disclosed optimism in the read path. Pricing it needs a decoder latency model on the read path and a 27% capacity cost applied to the density figures, and the latency and density verdicts then have to be re-derived together, because the two move in opposite directions.
- **Endurance-aware scheduling.** Wear-leveling algorithms at the controller level, simulated end to end rather than modeled analytically, against the finding of Section 4.3 that a single-region Start-Gap recovers nothing for these footprints.
- **Macro-scale 1T1R array limits.** The matched organization is a constraint on 1T1R rather than a description of its ceiling, and because chip leakage follows mat count, enlarging the 1T1R subarray is the most direct attack available on the peripheral-leakage floor, reporting leakage per gigabyte rather than latency as the primary metric.
- **A workload that would actually use rank depth.** None of the six traces asks for it: the CPU traces are offered-rate limited and the AI traces never leave the first rank span. A trace with a large address footprint *and* a sustained arrival rate above one controller's service rate would turn the strongest negative result of Section 4.4 into a measurement.

Data availability. The complete simulation stack, including the `mbmm_master.py` orchestration script, the modified NVSim and NVMain C++ sources, the generated JSON configurations and the frozen result datasets this book reports, is publicly available.

> **Project Repository:** https://github.com/uvKogan/Memristor-based-dram

The two simulators are git submodules of that repository, pinned at the patched NVMain fork `uvKogan/NVmain`, branch `ddr5-timing-fix`, commit `c89988d`, and the patched NVSim fork `uvKogan/MBMM`, commit `9a90d44`. A clone with `--recursive` checks out exactly the trees that produced the numbers in this book. The primary 120-run matrix is one invocation, 87 minutes on a standard Linux workstation; the extended book's Appendix C lists the sensitivity series and the standalone tools that produce the rest.

```
git clone --recursive https://github.com/uvKogan/Memristor-based-dram.git
cd Memristor-based-dram
python3 mbmm_master.py --all \
  --trace gcc_spec2017.nvt lbm_spec2017.nvt stream.nvt \
          gpt2_ifmap.nvt alexnet_layer1_ifmap.nvt alexnet_layer1_ofmap.nvt \
```

```
--window-ns 250000000 --ddr5-model DDR5_4800_DRAM_subchannel \
--channels 2 --decoder StartGap --endurance-model RowModel --silicon
```

References

[1] M. Ahmad, "NAND Flash's Reversal of Fortune Amid the AI Boom," *EE Times*, Jan. 26, 2026. [Online]. Available: https://www.eetimes.com/nand-flashs-reversal-of-fortune-amid-the-ai-boom/. [Accessed: Apr. 25, 2026].

[2] A. J. Fleischer, "How AI Broke the Memory Market," *Octopart Pulse*, Mar. 13, 2026. [Online]. Available: https://octopart.com/pulse/p/how-ai-broke-memory-market. [Accessed: Apr. 25, 2026].

[3] International Roadmap for Devices and Systems (IRDS), "More Moore," 2024. [Online]. Available: https://irds.ieee.org/images/files/pdf/2024/2024IRDS_MM.pdf.

[4] X. Dong et al., "NVSim: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 7, pp. 994-1007, July 2012.

[5] M. Poremba, T. Zhang, and Y. Xie, "NVMain 2.0: A user-friendly memory simulator to model (non-)volatile memory systems," IEEE Comput. Archit. Lett., vol. 14, no. 2, pp. 140-143, July-Dec. 2015.

[6] H.-S. P. Wong et al., "Metal-Oxide RRAM," *Proc. IEEE*, vol. 100, no. 6, pp. 1951-1970, June 2012. DOI: 10.1109/JPROC.2012.2190369

[7] C. Xue et al., "16.1 A 22nm 4Mb 8b-Precision ReRAM Computing-in-Memory Macro with 11.91 to 195.7TOPS/W for Tiny AI Edge Devices," *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, 2021, pp. 245-247.

[8] C. Matsui, A. Yamada, N. Misawa, and K. Takeuchi, "ReRAM resistance design of LRS and HRS for ultrahigh-capacity digital memory and analog Computation-in-Memory," IEICE Trans. Fundamentals, vol. E109.A, no. 3, pp. 596-603, Mar. 2026. DOI: 10.1587/transfun.2025VLP0011

[9] S. H. Jo et al., "3D-stackable crossbar resistive memory based on Field Assisted Superlinear Threshold (FAST) selector," *2014 IEEE International Electron Devices Meeting (IEDM)*, San Francisco, CA, USA, 2014, pp. 6.7.1-6.7.4.

[10] J. Zhou, K.-H. Kim, and W. Lu, "Crossbar RRAM Arrays: Selector Device Requirements During Read Operation," *IEEE Trans. Electron Devices*, vol. 61, no. 5, pp. 1369-1376, May 2014.

[11] A. Levy, L. R. Upton, M. D. Scott, D. Rich, W.-S. Khwa, Y.-D. Chih, M.-F. Chang, S. Mitra, B. Murmann, and P. Raina, "EMBER: Efficient Multiple-Bits-Per-Cell Embedded RRAM Macro for High-Density Digital Storage," *IEEE J. Solid-State Circuits*, vol. 59, no. 7, pp. 2081-2092, July 2024. DOI: 10.1109/JSSC.2024.3387566

[12] L. R. Upton et al., "EMBER: A 100 MHz, 0.86 mm2, multiple-bits-per-cell RRAM macro in 40 nm CMOS with compact peripherals and 1.0 pJ/bit read circuitry," in Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC), Sep. 2023, pp. 469-472.

[13] J. Zahurak et al., "Process Integration of a 27nm, 16Gb Cu ReRAM," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, Dec. 2014.

[14] T.-y. Liu et al., "A 130.7-mm² 2-Layer 32-Gb ReRAM Memory Device in 24-nm Technology," *IEEE J. Solid-State Circuits*, vol. 49, no. 1, pp. 140-153, Jan. 2014.

[15] Intel Corporation and Micron Technology, 3D XPoint technology announcement, July 2015. [Online]. Available: https://www.intc.com/news-events/press-releases/detail/324/intel-and-micron-produce-breakthrough-memory-technology.

[16] TechInsights, "Memory/Selector Elements for Intel Optane XPoint Memory." [Online]. Available: https://www.techinsights.com/blog/memoryselector-elements-intel-optanetm-xpoint-memory.

[17] D. Kau, S. Tang, I. V. Karpov, R. Dodge, B. Klehn, J. A. Kalb, J. Strand, A. Diaz, N. Leung, J. Wu, S. Lee, T. Langtry, K.-W. Chang, C. Papagianni, J. Lee, J. Hirst, S. Erra, E. Flores, N. Righos, H. Castro, and G. Spadini, "A Stackable Cross Point Phase Change Memory," in *2009 IEEE International Electron Devices Meeting (IEDM)*, Baltimore, MD, Dec. 2009, pp. 617-620. DOI: 10.1109/IEDM.2009.5424263

[18] Tom's Hardware, "Intel Kills Optane Memory Business Entirely, Pays $559 Million to Exit," Jul. 28, 2022. [Online]. Available: https://www.tomshardware.com/news/intel-kills-optane-memory-business-for-good. [Accessed: Jul. 13, 2026].

[19] J. Izraelevitz, J. Yang, L. Zhang, J. Kim, X. Liu, A. Memaripour, Y. J. Soh, Z. Wang, Y. Xu, S. R. Dulloor, J. Zhao, and S. Swanson, "Basic Performance Measurements of the Intel Optane DC Persistent Memory Module," arXiv:1903.05714, Aug. 2019.

[20] JEDEC Solid State Technology Association, "DDR5 SDRAM Standard," JESD79-5D, Nov. 2025. [Online]. Available: https://www.jedec.org/standards-documents/docs/jesd79-5d. Access-gated; the values used here were read from Micron Technology, "DDR5 SDRAM" core product data sheet, Doc. CCM005-1684161373-23, Rev. D, Oct. 2022, the vendor’s transcription of the standard, and checked against two DDR5-4800 module datasheets (extended book, reference [101]).

[21] B. C. Lee, E. Ipek, O. Mutlu, and D. Burger, "Architecting phase change memory as a scalable dram alternative," in *Proc. 36th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2009, pp. 2-13.

[22] Arm Ltd., "Neoverse N3 Technical Overview: Core Components" (64-byte cache lines). [Online]. Available: https://developer.arm.com/documentation/107997/0000/Technical-overview/Core-components.

[23] Avnet, "DDR5 Key Module Technical Brief" (two 32-bit subchannels at burst length 16, giving a 64-byte access). [Online]. Available: https://www.avnet.com/wcm/connect/630450bf-bb94-4993-b756-4b9f3ef2334e/DDR5-Key-Module-Technical-Brief.pdf.

[24] N. Binkert et al., "The gem5 Simulator," ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1-7, May 2011. DOI: 10.1145/2024716.2024718

[25] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A Systematic Methodology for Characterizing Scalability of DNN Accelerators Using SCALE-Sim," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2020, pp. 58-68.

[26] Standard Performance Evaluation Corporation, "SPEC CPU 2017 Benchmark Suite." [Online]. Available: https://www.spec.org/cpu2017/. [Accessed: Jul. 13, 2026].

[27] J. D. McCalpin, "Memory Bandwidth and Machine Balance in Current High Performance Computers," IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, pp. 19-25, Dec. 1995.

[28] Program-and-verify pulse-count characterization of multi-level-cell 1T1R RRAM, preprint, arXiv:2108.07879 (average 49 pulses per level move, median 36).

[29] Program-and-verify characterization of multi-level-cell phase-change memory (more than 70 iterations for a 9.7 us total write; under 1.6 us with the verify step skipped), PMC6680789. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC6680789/.

[30] K. T. Malladi, F. A. Nothaft, K. Periyathambi, B. C. Lee, C. Kozyrakis, and M. Horowitz, "Towards Energy-Proportional Datacenter Memory with Mobile DRAM," in Proc. 39th Annu. Int. Symp. Computer Architecture (ISCA), 2012, pp. 37-48. DOI: 10.1109/ISCA.2012.6237004

[31] Y. Chen, "ReRAM: History, Status, and Future," *IEEE Trans. Electron Devices*, vol. 67, no. 4, pp. 1420-1433, Apr. 2020.

[32] M. K. Qureshi, J. Karidis, M. Franceschini, V. Srinivasan, L. Lastras, and B. Abali, "Enhancing Lifetime and Security of PCM-Based Main Memory with Start-Gap Wear Leveling," in *Proc. 42nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2009, pp. 14-23.

[33] Measured idle latency of local and remote-socket DDR5 and of CXL-attached DRAM devices, preprint, arXiv:2409.14317.

[34] "Pond": CXL-based memory pooling for cloud platforms (Microsoft Azure), preprint, arXiv:2203.00241 (local DDR5 platform baselines, pooling latency overheads, workload latency-acceptability study).

[35] Sun et al., "Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices," preprint, arXiv:2303.15375.

[36] Linux kernel documentation, "NUMA performance" (ACPI HMAT latency and bandwidth attributes exposed per initiator-target pair). [Online]. Available: https://docs.kernel.org/admin-guide/mm/numaperf.html.

[37] Micron Technology, "Micron launches memory expansion module portfolio to accelerate CXL 2.0 adoption," Aug. 7, 2023 (CZ120, x8, up to 36 GB/s). [Online]. Available: https://www.globenewswire.com/news-release/2023/08/07/2719764/14450/en/Micron-Launches-Memory-Expansion-Module-Portfolio-to-Accelerate-CXL-2-0-Adoption.html.

[38] Astera Labs, CM5162 Leo smart memory controller product page (CXL 1.1 / 2.0, x16 at 32 GT/s, two DDR5 channels, up to 2 TB; no published latency figure). [Online]. Available: https://www.asteralabs.com/product-details/cm5162p/.

[39] Intel Corporation, second-generation Intel Xeon Scalable processor family technical overview (Optane PMem Memory Mode and App Direct; 64-byte byte-addressable lines). [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/news/second-generation-intel-xeon-processor-scalable-family-technical-overview.html.

[40] Channel-model bit error rates for a 1S1R cross-point array (1024 x 1024, 10 ohm per junction, ideal selector; read and write error rates by cell position), preprint, arXiv:1912.02963.

[41] Weebit Nano, smart-write algorithm technical resource (128 kb 1T1R macro: below 10^{-7} after 10^{6} cycles, 28.1 dB read margin). [Online]. Available: https://www.weebit-nano.com/technical_resources/smart-write-algorithm-to-enhance-performances-and-reliability-of-an-rram-macro/.

[42] Raw bit error rate of 1T1R HfO_2 RRAM arrays under cycling, including a 2T2R variant, preprint, arXiv:1902.02528.

[43] Reliability of a 1S1R selector stacked with STT-MRAM (10^{-9} per-read disturb rate, 10^{9} read-disturb-free cycles), IEDM 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10413809/.

[44] IBM Research, "Enhancing 3D XPT/SOM reliability: strategies for mitigating spike current and improving read endurance," IMW 2025 (cell failure rate 9% falling to 0.02% after read-endurance optimization). [Online]. Available: https://research.ibm.com/publications/enhancing-3d-xptsom-reliability-strategies-for-mitigating-spike-current-and-improving-read-endurance.

[45] Zhang, Sridharan, and Jian, chipkill-correct for persistent memory, *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2018; consulted via ar5iv:2309.00304 (2 x 10^{-4} raw design point, BCH(2312,2048,22), about 27% coding overhead).

[46] ArchShield, an architectural framework for DRAM with high bit-error rates; consulted via ar5iv:1704.03991 (DRAM on-die SECDED tolerates about 10^{-6} raw).

[47] Intel Corporation, Intel Optane SSD 905P series product specifications (uncorrectable bit error rate under 1 sector per 10^{17} bits). [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/147526/intel-optane-ssd-905p-series-1-5tb-2-5in-pcie-x4-3d-xpoint/specifications.html.

[48] Read-margin analysis of 1R against 1S1R crossbar arrays, establishing the 10% minimum read-margin criterion, arXiv:2407.10466.